Algorithms for Deciding the Safety of States in Fully Observable Non-deterministic Problems: Technical Report

Johannes Schmalz, Chaahat Jain

Saarland University, Germany

Abstract

Learned action policies are increasingly popular in sequential decision-making, but suffer from a lack of safety guarantees. Recent work introduced a pipeline for testing the safety of such policies under initial-state and action-outcome non-determinism. At the pipeline's core, is the problem of deciding whether a state is safe (a safe policy exists from the state) and finding *faults*, which are state-action pairs that transition from a safe state to an unsafe one. Their most effective algorithm for deciding safety, TarjanSafe, is effective on their benchmarks, but we show that it has exponential worst-case runtime with respect to the state space. A linear-time alternative exists, but it is slower in practice. We close this gap with a new *policy-iteration* algorithm iPI, that combines the best of both: it matches TarjanSafe's best-case runtime while guaranteeing a polynomial worst-case. Experiments confirm our theory and show that in problems amenable to TarjanSafe iPI has similar performance, whereas in ill-suited problems iPI scales exponentially better.

Short Version Published at ICAPS 2026 —
https://doi.org/10.1609/icaps.v36i1.42858

1 Introduction

Learned action policies are gaining traction in AI and AI planning, e.g., Mnih et al. (2015); Silver et al. (2016, 2018); Toyer et al. (2020). However, such policies come without any safety guarantees, creating demand for safety assurance methods. We focus on the safety-testing framework of Jain et al. (2025) (henceforth JEA) in fully observable non-deterministic (FOND) planning. In JEA's setting, a policy π is **safe** in a given state s if its execution will never lead to a **failure condition** ϕ_F, a state is **safe** if it has a safe policy, and **faults** are state-action pairs $(s, \pi(s))$ where s is safe but applying $\pi(s)$ non-deterministically leads to an unsafe state s'. Their framework takes a learned policy, finds paths therein that lead to fail states, and then finds faults by deciding the safety of states along the paths. These faults can be used to repair the policy. To decide whether a state is safe JEA consider two orthogonal approaches: (1) adapting well-known algorithms for Markov Decision Processes (MDPs), such as Value Iteration (VI), LAO* (Hansen and Zilberstein 2001), and LRTDP (Bonet and Geffner 2003); (2) creating a specialised Depth-First Search (DFS) with a mechanism for detecting Strongly Connected Components (SCCs), which they call TarjanSafe. Their results showed that TarjanSafe is very effective for deciding safety.

In this paper, we show that TarjanSafe has an *exponential worst-case time complexity w.r.t. the state space* using a pathological family of tasks where TarjanSafe expands the same states many times. Despite this limitation, TarjanSafe performs well in practice because it often explores only a small fraction of the state space. To contrast, we present Prop-$\mathcal{U}$, which follows techniques from reachability games (Diekert and Kufleitner 2022) and works by propagating unsafe states. Its worst case is linear, but it is unusable in practice because there are too many unsafe states to propagate. Prop-$\mathcal{U}$ is unusable in practice because there are too many unsafe states to propagate. To address this gap we introduce iPI, a new algorithm based on **policy iteration** that combines a *polynomial worst-case runtime* while enjoying the practical speed of TarjanSafe; iPI starts with an initial policy π_{init} and attempts to prove its safety, making minimal changes to the policy as unsafe states are discovered; this makes iPI very efficient at finding safe policies similar to π_{init}, which occurs frequently. Finally, we confirm our theory experimentally. We use JEA's benchmarks, which mostly correspond to TarjanSafe's best case; there, iPI shows similar performance to TarjanSafe. Then, we introduce the *Flappy Bird* domain to showcase a problem ill-suited to TarjanSafe, and indeed iPI scales exponentially better. This places iPI as the current best algorithm for deciding state safety and gives valuable insight into what makes a good algorithm for this problem.

Our contributions are: a simplified formalism for FOND safety, new algorithms for deciding safety (Prop-$\mathcal{U}$ and iPI), a complexity analysis of TarjanSafe and our algorithms, and an empirical evaluation that shows iPI's effectiveness.

2 Background

We consider fully-observable non-deterministic (FOND) planning tasks (Daniele, Traverso, and Vardi 2000; Cimatti et al. 2003). We follow the formalism of JEA, but break it down into a simpler, equivalent formulation. A task's transition system is defined by $\Theta = \langle \mathsf{S}, \mathsf{A}, \mathcal{T}, \mathsf{S}_I \rangle$ where

- S is a finite set of states,
- A is a finite set of actions and we write $\mathsf{A}(s)$ to denote

the set of actions applicable in state s,

- $\mathcal{T}$ is a non-deterministic transition function where $\mathcal{T}(s,a) \in \mathcal{P}(\mathsf{S}) \setminus \emptyset$ gives the set of states that may occur after applying a to s,
- $\mathsf{S}_I \subsetneq \mathsf{S}$ is a non-empty set of initial states.

Policies are partial functions $\pi : \mathsf{S} \to \mathsf{A}$ with the property that $\pi(s) \in \mathsf{A}(s)$ wherever defined. A policy π induces a **policy graph** $\Theta^\pi = \langle \mathsf{S}, \mathsf{A}, \mathcal{T}^\pi, \mathsf{S}_I \rangle$ with the same components as Θ except $\mathcal{T}^\pi$ has $\mathcal{T}^\pi(s,a) = \mathcal{T}(s,a)$ if $a = \pi(s)$ and is undefined otherwise. A **path** is a finite or infinite sequence of alternating states and actions $\sigma = s_0, a_0, s_1 \dots$ that satisfies $a_i \in \mathsf{A}(s_i)$ and $s_{i+1} \in \mathcal{T}(s_i, a_i)$ for all $i \geq 0$ in σ. The set of all paths from a state s is *Paths*(s) and *Paths*$^\pi(s)$ is the set of paths from s in the policy graph Θ^π.

Typically, FOND tasks are also specified with a non-empty set of goal states $\mathsf{G} \subseteq \mathsf{S}$. We call such tasks $\Theta = \langle \mathsf{S}, \mathsf{A}, \mathcal{T}, \mathsf{S}_I, \mathsf{G} \rangle$ **goal-reaching FOND tasks**. To solve them we consider strong cyclic solutions, which we present as policies π s.t. following π from any initial state $s_I \in \mathsf{S}_I$ will eventually reach a goal in G assuming *fairness*.

Assumption 1 (Fairness)**.** *If an action a is applied infinitely-many times in state s, then each non-deterministic effect in $\mathcal{T}(s,a)$ must trigger infinitely often (Cimatti et al. 2003).*

Formally, π solves a goal-reaching FOND task if all finite-length paths in $\bigcup_{s_I \in \mathsf{S}_I}$ *Paths*$^\pi(s_I)$ terminate in G.

Recall that our setting is to identify whether a safe policy exists for states in a learned policy's envelope, so in this paper we focus on avoiding certain fail states rather than goal-reaching. Thus, we specify a non-empty set of **fail states** $\mathsf{S}_f \subseteq \mathsf{S}$, giving us **state-avoiding FOND tasks** $\Theta = \langle \mathsf{S}, \mathsf{A}, \mathcal{T}, \mathsf{S}_I, \mathsf{S}_f \rangle$.[1] A solution to such Θ is a policy π that never encounters any fail state, i.e., for all finite and infinite paths $\sigma \in \bigcup_{s_I \in \mathsf{S}_I}$ *Paths*$^\pi(s_I)$, σ does not contain any states in S_f. State-avoiding FOND lets us express notions of safety, i.e., that a "bad" state should never happen, e.g., a self-driving car must never crash into a wall. A state s is called **unsafe** if no policy can guarantee that it avoids fail states from s, i.e., $\forall \pi \; \exists \sigma \in$ *Paths*$^\pi(s) : \sigma \cap \mathsf{S}_f \neq \emptyset$. The set of unsafe states is called the unsafe region $\mathcal{U}$.

Whether a policy π is state-avoiding (i.e., safe) can be evaluated with the following *value function*.

Definition 1. *The value of policy π at state s is*

$$V^\pi(s) = \begin{cases} 1 & \text{if } s \in \mathsf{S}_f \\ \max_{s' \in \mathcal{T}(s,\pi(s))} V^\pi(s') & \text{if } s \notin \mathsf{S}_f. \end{cases}$$

The policy π is safe from a state s, i.e., there is a way to avoid unsafe states, iff $V^\pi(s) = 0$.

$V^*(s)$ is the optimal value at s, given by $V^*(s) = \min_{\pi \in \Pi} V^\pi(s)$. Given V^*, if a safe policy exists from s, then one can extract a safe policy by following V^* greedily. V^* can be computed with Value Iteration (VI). VI starts with an initial value function, e.g., $V(s) = [\![s \in \mathsf{S}_f]\!] \; \forall s \in \mathsf{S}$, and then applies Bellman backups:

$$V(s) \leftarrow \min_{a \in \mathsf{A}(s)} \max_{s' \in \mathcal{T}(s,a)} V(s').$$

[1] JEA use a failure condition ϕ_F, but for our discussion it is equivalent to consider a specified set of fail states $\mathsf{S}_f \subseteq \mathsf{S}$.

The term being minimised in the Bellman backup is often called a Q-value, so for us $Q(s,a) = \max_{s' \in \mathcal{T}(s,a)} V(s')$. By repeatedly applying Bellman backups over all states the value function will eventually converge to V^*. JEA give arguments for this, and it is also demonstrated by Prop-$\mathcal{U}$ later.

Dually to $V^*(s) = 0$ indicating a safe state, $V^*(s) = 1$ indicates that s is an unsafe state, from which there is no policy that can guarantee that it avoids fail states S_f.

Now, we return to the problem formalisation. JEA look for safe policies in tasks that have goals, other terminal states, and fail states to avoid. Their policies must avoid the fail states, either by reaching a terminal state and stopping there, or by cycling in a way that avoids the fail states. JEA's tasks can be transformed into our state-avoiding tasks by adding a self-loop action to goal and terminal states, allowing policies to avoid fail states indefinitely by using these self loops rather than terminating. For the rest of this paper we focus on deciding the safety of individual states, so we only consider singleton initial states $\mathsf{S}_I = \{s_I\}$.

We note that state-avoiding tasks can be cast into goal-reaching ones. Given a state-avoiding problem $\Theta = \langle \mathsf{S}, \mathsf{A}, \mathcal{T}, \mathsf{S}_I, \mathsf{S}_f \rangle$, we can define an equivalent goal-reaching problem $\Theta' = \langle \mathsf{S} \cup \{g\}, \mathsf{A}, \mathcal{T}', \mathsf{S}_I, \{g\} \rangle$. This introduces the artificial goal g and all transitions from non-fail states gain g as a non-deterministic effect from all safe states; also, we turn fail states into "traps" that only loop back to themselves and can never reach the goal. Formally, $\forall s \in \mathsf{S}, a \in \mathsf{A}(s)$

$$\mathcal{T}'(s,a) = \begin{cases} \mathcal{T}(s,a) \cup \{g\} & \text{if } \mathcal{T}(s,a) \text{ defined and } s \notin \mathsf{S}_f \\ \{s\} & \text{if } \mathcal{T}(s,a) \text{ defined and } s \in \mathsf{S}_f \\ \text{undefined} & \text{if } \mathcal{T}(s,a) \text{ undefined.} \end{cases}$$

Now, Θ' has a policy that always reaches its goal $\{g\}$ iff Θ has a policy that avoids the states S_f. This is clear because a goal-reaching policy for Θ' can not enter any fail state S_f, and therefore describes a state-avoiding policy for Θ. Going the other direction, a state-avoiding policy for Θ never enters S_f, and so all its actions have g as one of the effects, which means that, due to the fairness assumption (assump. 1), the policy must eventually reach the artificial goal in Θ'.

This transformation from state-avoiding to goal-reaching lets us solve the state-avoiding problem in terms of goal-reaching with state-of-the-art FOND planners such as PR2 (Muise, McIlraith, and Beck 2024). However, we limit the scope of this paper to fundamental algorithms that are native to state-avoiding, letting us make a fairer comparison to JEA and perform sharper complexity analyses, thereby establishing a foundation for future work. Moreover, using their algorithms in our codebase is non-trivial because it requires a translation from JANI to variants of PDDL, which is an open line of research, e.g., (Klauck et al. 2018).

3 Algorithms

We first explain TarjanSafe (JEA) and prove that it has a worst-case exponential runtime w.r.t. the state-space, but a good best-case that reflects its strong performance in practice. Then, we show Prop-$\mathcal{U}$, an algorithm that solves state-avoiding tasks in linear time, but has a worse best-case that makes it impractical. Finally, we introduce a Policy Iteration

	Best case	Worst case
TarjanSafe (alg. 1)	$\Theta(\lvert\pi_{\text{min-safe}}\rvert)$	$\Omega(2^m)$
Prop-$\mathcal{U}$ (alg. 2)	$\Theta(\lvert\mathsf{S}_f\rvert)$	$\Theta(bm)$
nPI (alg. 3), iPI (alg. 4)	$\Theta(\lvert\pi_{\text{min-safe}}\rvert)$	$O(bmn)$

Table 1: Best-case (where a safe policy exists) and worst-case runtimes of the algorithms we consider. We use $m = |\mathsf{A}|$, $n = |\mathsf{S}|$, branching factor of non-determinism b, and $|\pi_{\text{min-safe}}|$ is a minimal safe policy's envelope size.

algorithm and prove that it combines a polynomial worst case with the same best case as TarjanSafe. We use the notation $m = |\mathsf{A}|$, $n = |\mathsf{S}|$, $b = \max_{s\in\mathsf{S},a\in\mathsf{A}(s)} |\mathcal{T}(s,a)|$ denotes a bound on the branching factor of non-determinism, and $\pi_{\text{min-safe}}$ is a safe policy with a minimal number of states in its policy graph. We assume that $\mathsf{A}(s) \neq \emptyset$ for each state, so $m \geq n$. Our complexity results are summarised in tab. 1. We restrict the best-case runtime analyses to tasks where a safe policy exists; if no safe policy exists then all the algorithms we consider can be implemented with an early stop, in which case they terminate in one or two steps, i.e., $O(1)$.

3.1 TarjanSafe

TarjanSafe is JEA's algorithm for finding FOND state-avoiding policies. It is a DFS with a labelling mechanism for states that have been proven safe or unsafe, and it has a mechanism for detecting SCCs in its candidate policy using the stack and lowlink indices from Tarjan's algorithm (Tarjan 1972), hence the name TarjanSafe. We present pseudocode for the algorithm in alg. 1.[2] TarjanSafe's DFS is defined recursively: in its recursive cases, TarjanSafe iterates over $a \in \mathsf{A}(s)$ and calls itself on the successor states $\mathcal{T}(s,a)$, then it either marks s as unsafe if all these actions lead to unsafe states, or otherwise the algorithm assumes that s is safe and proceeds. Additionally, TarjanSafe detects SCCs and marks the SCC's root as safe if the SCC contains no unsafe states. Then, the recursion stops at s without making further recursive calls in the following cases:

1. s is a fail state ($s \in \mathsf{S}_f$),
2. s marked as unsafe with `known-unsafe`$[s]$,
3. s is a goal state (not relevant in our setting),
4. s marked as safe with `known-safe`$[s]$,
5. some action in $\mathsf{A}(s)$ leads only to states in the stack ($\mathcal{T}(s,a) \subseteq$ `stack`).

We motivate TarjanSafe's practical effectiveness with its best-case runtime.

Theorem 1. *TarjanSafe has a best-case runtime of* $\Theta(|\pi_{\text{min-safe}}|)$ *(if a safe policy exists).*

Proof. Suppose $\pi_{\text{min-safe}}$ is a safe policy with minimal policy graph such that the policy graph is a tree, except its leaf nodes which have self-loops. Further suppose that TarjanSafe guesses $\pi_{\text{min-safe}}$, i.e., in its DFS it always selects $\pi_{\text{min-safe}}(s)$ for s. Then, the algorithm traverses $\pi_{\text{min-safe}}$ precisely once with its DFS to prove that the states are safe, and does not need to do any branching nor backtracking on the actions choices, so it is $O(|\pi_{\text{min-safe}}|)$. To verify that a policy π is safe, one must at least check that each state in its policy graph is not a fail state S_f, so it is impossible to do better than $\Omega(|\pi_{\text{min-safe}}|)$. Thus, TarjanSafe's best-case must be $\Theta(|\pi_{\text{min-safe}}|)$. □

[2] JEA present a more general algorithm where a *safety radius* r can be specified, for us $r = \infty$. Also, we assume our state-avoiding formulation. Our presentation has been simplified accordingly.

Algorithm 1: JEA's TarjanSafe

```
1  Function TarjanSafe(Θ, s_I)
2      known-safe[s] ← false ∀s ∈ S
3      known-unsafe[s] ← false ∀s ∈ S
4      stack ← empty stack
5      low ← empty map
6      return rec-TarjanSafe(s_I)
7  Function rec-TarjanSafe(s)
8      // Has access to TarjanSafe's vars
9      if s ∈ S_f or known-unsafe[s] then
10         return false
11     if known-safe[s] then
12         return true
13     stack-idx ← ‖stack‖
14     push s onto stack
15     low[s] ← stack-idx
16     A-unsafe ← true
17     foreach a ∈ A(s) and while A-unsafe do
18         a-safe ← true
19         foreach s' ∈ T(s,a) and while a-safe do
20             if s' ∈ stack then
21                 low[s] ← min(low[s], low[s'])
22             else
23                 a-safe ← rec-TarjanSafe(s')
24         A-unsafe ← ¬a-safe
25     if A-unsafe then
26         known-unsafe[s] ← true
27     else
28         // Update safe at root of SCC
29         if low[s] = stack-idx then
30             known-safe[s] ← true
31             pop from stack all states down to s
32     pop s from stack
33     return ¬A-unsafe
```

But in the worst case, TarjanSafe is exponential.

Theorem 2. *TarjanSafe has a worst-case runtime of* $\Omega(2^m)$.

Proof. We present a class of problems where TarjanSafe requires on the order of 2^m steps, which proves that TarjanSafe's worst-case is asymptotically 2^m or worse, i.e., $\Omega(2^m)$. Consider the task in fig. 1 with 5 layers, and consider the generalised construction with d layers. This task has no fail, unsafe, nor goal states, so TarjanSafe's cases 1–3 will never trigger for any state. Case 4 can only be triggered at an SCC's root, which is only s_0, so this case only triggers when the algorithm terminates. Thus, the recursion can only

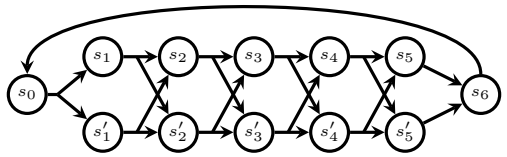

Figure 1: Non-deterministic task with 5 layers and 2^5 paths from s_0 to s_6. Similarly constructed tasks with d layers have 2^d paths from s_0 to s_{d+1}.

bottom out at state s if s's applicable action leads to states that are already on the `stack`, which in turn only happens at the state in the last layer, leading to s_0. This forces the algorithm to re-expand states and enumerate all paths. In our task with d layers, TarjanSafe must enumerate all paths from s_0 to s_{d+1} before it can terminate, which is 2^d paths. The problem has $2d+2$ states and $2d+2$ actions with a branching factor b of 2, so indeed the runtime of the algorithm is exponential w.r.t. the transition system size. □

We now show that with other approaches our problem can in fact be solved in polynomial time, even in the worst case.

3.2 Unsafety Propagation

We define Prop-$\mathcal{U}$, and show that it solves state-avoiding tasks in linear time by propagating the unsafe region. This result draws from reachability games — such games have two players who alternate turns: one player must reach certain states and their opponent must make sure that these states are never reached. This framework is immediately analogous to our setting where the fail states S_f correspond to the winning states of an "unsafety player," and the "safety player" wins by ensuring that the unsafety player never wins — a winning strategy for the safety player corresponds to a safe policy for us. The unsafety player can be understood as an adversarial choice of non-deterministic outcomes.[3] The existence of a linear-time algorithm is considered folklore for reachability games, e.g., (Mazala 2002, Exercise 2.6). Diekert and Kufleitner (2022) present a concrete linear-time algorithm for solving reachability games.

In alg. 2, we define Prop-$\mathcal{U}$ by adapting their algorithm to our state-avoiding FOND setting and formulating it in the framework of Value Iteration. Effectively, Prop-$\mathcal{U}$ is propagating the unsafe region from the known unsafe states (initially only the fail states), and then we know that a safe policy from s_I exists iff s_I is outside the unsafe region. Thus, upon termination, $V(s) = 1$ iff s is unsafe. Be aware that throughout execution the semantic of V is slightly different: we have that $V(s) = 1$ iff s is proved unsafe, and $V(s) = 0$ indicates that we have not proved s either way.

Theorem 3. *Prop-$\mathcal{U}$ (alg. 2) returns* 0 *if* s_I *is safe and* 1 *if* s_I *is unsafe.*

[3]FOND fairness is not violated with this interpretation, since the unsafety player will never voluntarily choose to get trapped in a cycle, as they are trying to reach unsafe states in finite steps.

Algorithm 2: Propagate Unsafe States

Function *Prop-$\mathcal{U}$*(Θ, s_I)
  `// Here` Q `is a separate variable, not a function of` V
  $V(s) \leftarrow [\![s \in \mathsf{S}_f]\!] \; \forall s \in \mathsf{S}$
  $Q(s,a) \leftarrow 0 \; \forall s \in \mathsf{S}, a \in \mathsf{A}(s)$
  $\mathcal{Z} \leftarrow \mathsf{S}_f$
  while $\mathcal{Z} \neq \emptyset$ **do**
    $s' \leftarrow \mathcal{Z}$.pop()
    for $(s,a) : s' \in \mathcal{T}(s,a)$ **do**
      $Q(s,a) \leftarrow 1$
      if $V(s) = 0$ *and* $\min_{a' \in \mathsf{A}(s)} Q(s,a') = 1$ **then**
        $V(s) \leftarrow 1$
        $\mathcal{Z} \leftarrow \mathcal{Z} \cup \{s\}$
  return $V(s_I)$

Proof. We want to show that a state s updates $V(s) \leftarrow 1$ and enters $\mathcal{Z}$ iff it is unsafe. Only unsafe states enter $\mathcal{Z}$ by an inductive argument: fail states S_f are the base case, and otherwise s only enters $\mathcal{Z}$ if all its successors are unsafe with $\min_{a' \in \mathsf{A}(s)} Q(s,a') = 1$. It remains to show that all unsafe states will eventually enter $\mathcal{Z}$. For contradiction, suppose that an unsafe state does not enter $\mathcal{Z}$, which implies that there is an unsafe state s with $V(s) = 0$. This state must have an action $a \in \mathsf{A}(s)$ that leads to an unsafe state s' but has $Q(s,a) = 0$. Consequently, we can select an unsafe state $s' \in \mathcal{T}(s,a)$ with $V(s') = 0$. Then we can apply the same argument to s', and repeat this argument to produce a chain $\langle s_0, s_1, \dots \rangle$ of unsafe states with $V(s_i) = 0$ for all s_i in the chain. By selecting *some action* with $Q(s,a) = 0$ we are describing a policy, and since each state we are considering is unsafe there must be a path in this policy graph that leads to a fail state. So, it must be possible to extend our chain so that it eventually reaches a fail state, but fail states have $V(s) = 1$, yielding the desired contradiction. □

Theorem 4. *Prop-$\mathcal{U}$ (alg. 2) has a runtime of* $\Omega(|\mathcal{Y}|)$ *and* $O(b|\mathcal{Y}|)$ *where* $\mathcal{Y} = \{(s,a) : s \in \mathsf{S}, a \in \mathsf{A}(s), \mathcal{T}(s,a) \cap \mathcal{U} \neq \emptyset\}$*, i.e.,* $\mathcal{Y}$ *are the state-action pairs with an effect in the unsafe region.*

Proof. Each unsafe state $s \in \mathcal{U}$ will get added to $\mathcal{Z}$ precisely once (as argued before), so the algorithm's inner *for* loop will precisely iterate over $\mathcal{Y}$ with some potential repetitions if $(s,a) \in \mathcal{Y}$ has multiple unsafe states in $\mathcal{T}(s,a)$. This gives $\Omega(|\mathcal{Y}|)$. To handle the repetitions, we observe that $\mathcal{T}(s,a)$ can not contain more unsafe states than the non-deterministic branching factor b. Thus, we iterate over $\mathcal{Y}$ with at most b duplicates, i.e., the algorithm is $O(b|\mathcal{Y}|)$. □

Corollary 5. *Prop-$\mathcal{U}$ (alg. 2) has a best-case runtime of* $\Theta(|\mathsf{S}_f|)$ *(if a safe policy exists) and a worst-case runtime of* $\Theta(bm)$.

If no safe policy exists from the initial state s_I then Prop-$\mathcal{U}$ can stop as soon as s_I enters $\mathcal{Z}$, which may happen in a single step.

We call Prop-$\mathcal{U}$'s worst-case runtime of $\Theta(bm)$ linear, even though it contains two terms. Recall that b corresponds to the non-deterministic branching factor, and m is the number of actions, so bm gives the number of transitions $\langle s, a, s' \rangle$ in our state space. In that sense, Prop-$\mathcal{U}$ is linear w.r.t. the number of transitions. Connecting to reachability games, bm corresponds to the number of edges in an equivalent game: the algorithm of Diekert and Kufleitner (2022) is linear over the number of edges in a game, so it is reasonable to consider our adaptation Prop-$\mathcal{U}$ linear in our setting.

Since Prop-$\mathcal{U}$ is in the VI framework, we can compare it to standard VI and variants like Topological VI (TVI) (Dai et al. 2011). Prop-$\mathcal{U}$ dominates VI and TVI in the sense that in its worst case it computes $Q(s, a)$ once for each $s \in \mathsf{S}, a \in \mathsf{A}(s)$, whereas VI and TVI perform the same computations in their best case, and usually more.

Prop-$\mathcal{U}$ has the lowest worst-case complexity of the algorithms we consider in this paper, and it seems unlikely that it is possible to do better. However, in our setting, the unsafe region and therefore $\mathcal{Y}$ is very large (exponential w.r.t. problem description). This makes Prop-$\mathcal{U}$ impractical in practice. It is possible to exploit problem structure and propagate the set of unsafe states with Binary Decision Diagrams (BDDs), as explained in appendix A. However, propagating unsafe states with BDDs still fails in our problems, because the BDDs explode in size after only a few steps of propagation. It seems the approach of propagating unsafe states is fundamentally unsuitable to our problems.

In the next section, we present an alternative algorithm that combines the best of TarjanSafe and Prop-$\mathcal{U}$: it can stop as soon as it finds a safe policy but maintains a polynomial worst-case runtime.

3.3 Policy Iteration (PI)

Our implementation of PI in alg. 3, named nPI, makes a nice tradeoff: it has the same best case as TarjanSafe and remains polynomial in the worst case. PI is well-known as an MDP algorithm (Howard 1960), and arguably, successful goal-reaching FOND algorithms such as PR2 (Muise, McIlraith, and Beck 2024) fit within its framework. In our setting, the idea of PI is that it starts with some policy π, determines where π is unsafe, fixes π, and repeats until π is safe or s_I is proven unsafe. We define nPI using a Value Function V with the semantic that $V(s) = 1$ if we know that s is unsafe, and $V(s) = 0$ if we are unsure (as in Prop-$\mathcal{U}$). Initially, we only know that the fail states are unsafe so $V(s) \leftarrow [\![s \in \mathsf{S}_f]\!]$ (line 2). We use the shorthand $Q(s, a) = \max_{s' \in \mathcal{T}(s,a)} V(s')$, which is 1 if we know it is unsafe to apply a in s, and 0 if we are unsure. Then, we propagate the states that we know are unsafe by updating $V(s)$ with its minimal Q-value (line 12).

We now analyse the runtime complexity of nPI.

Theorem 6. *nPI (alg. 3) has a best-case runtime of $\Theta(\pi_{\textit{min-safe}})$ (if a safe policy exists).*

Proof. The argument is similar to the proof of thm. 1. In the best case, π_{init} is $\pi_{\text{min-safe}}$, so that nPI only needs to perform a DFS over π_{init} once to evaluate the policy. □

Algorithm 3: Naïve Policy Iteration

```
1  Function nPI(Θ, s_I, π_init)
2      V(s) ← [[s ∈ S_f]] ∀s ∈ S
3      π ← π_init
4      repeat
5          V, saw-unsafe ← Mark-Unsafe(Θ, s_I, V, π)
6          if ¬saw-unsafe or V(s_I) = 1 then
7              return V(s_I)
8          π ← Greedy(Θ, s_I, V)
9  Function Mark-Unsafe(Θ, s_I, V, π)
10     E ← DFS post order of policy graph Θ^π from s_I
11     for s ∈ E do
12         V(s) ← min_{a∈A(s)} Q(s, a)
13     saw-unsafe ← [[∃s ∈ E s.t. V(s) = 1]]
14     return V, saw-unsafe
15 Function Greedy(Θ, s_I, V)
16     new π undefined everywhere
17     while π has undefined state reachable from s_I do
18         s ← undefined state reachable from s_I
19         π(s) ← argmin_{a∈A(s)} Q(s, a)
20     return π
```

If no safe policy exists then nPI can stop as soon as it propagates the unsafe region to s_I and sets $V(s_I) \leftarrow 1$. With an early-stop mechanism (as implemented later in iPI), this may happen in as few as two steps.

Theorem 7. *nPI (alg. 3) has a worst-case runtime of $O(bmn)$.*

Proof. The functions *Mark-Unsafe* and Greedy are both $O(bm)$ because they are implemented with a variant of DFS that does not re-expand states, i.e., it expands each state at most once. It remains to show that the main loop (lines 4–8) repeats at most n times. To this end we argue that for each pass of the loop where π remains unsafe, at least one state with $V(s) = 0$ gets updated with $V(s) \leftarrow 1$; this can happen at most n times, because before then we either get a safe policy and terminate, or we set $V(s_I) = 1$ and terminate. Suppose we have computed $\pi \leftarrow$ *Greedy*(Θ, s_I, V) and V, π-is-safe $\leftarrow$ *Mark-Unsafe*(Θ, s_I, V, π), and neither of the termination conditions are satisfied, i.e., π is unsafe and $V(s_I) = 0$. We know that $V(s_I) = 0$, since π is unsafe its envelope must contain at least one state $s^\dagger$ with $V(s^\dagger) = 1$, and there is a (potentially empty) sequence of intermediate states between s_I and $s^\dagger$. On that sequence, there must be a pair of states s_i with $V(s_i) = 0$ and $V(s_{i+1}) = 1$ by a discrete intermediate value argument, i.e., if $V(s)$ can only be 0 or 1, and at one end of the sequence we have $V(s_I) = 0$ and at the other $V(s^\dagger) = 1$, then somewhere it has to switch from 0 to 1. Consequently, $Q\big(s_i, \pi(s_i)\big) = 1$. Recall that π was constructed greedily, so $\min_{a \in \mathsf{A}(s_i)} Q(s, a) = 1$ which means that $V(s_i)$ must be updated to 1, concluding the proof. □

The implementation in alg. 3 has various inefficiencies which we address in the **Improved Policy Iteration** algo-

rithm iPI (alg. 4). The key change in iPI is that the construction of a greedy policy and its evaluation are combined into a single step. Consequently, the algorithm only needs to traverse the candidate policy's envelope once per iteration, rather than twice. Moreover, it enables us to stop the construction of the greedy policy as soon as an unsafe state has been detected in its envelope (alg. 4 line 13); this avoids the expensive situation of nPI, where *Greedy*$(\cdot\cdot)$ encounters an unsafe state early on but continues construction, even though the policy can not be safe.

In the change from nPI to iPI we also generalise the notion of an initial policy into an action order $\mathbb{A}$ where $\mathbb{A}(s)$ returns the same actions as $\mathsf{A}(s)$ but ordered. Consequently, we do not provide π_{init}, as it is captured by the first "preference" of each $\mathbb{A}(s)$. This leads to the subtle detail that we no longer construct policies that are greedy w.r.t. V, but rather we construct policies according to the preferences of $\mathbb{A}$. We made this choice because profiling revealed that computing Q-values to compute the greedy policy w.r.t. V is relatively expensive, and does not pay off, i.e., using an arbitrary fixed order over actions had better performance. The choice of $\mathbb{A}$ can be important, since a good ordering may quickly lead to a safe policy or to a way to prove s_I as unsafe, and a bad one causes iPI to waste time.

We note that with iPI's recursive definition it is not immediately clear whether the main loop (lines 3–8) is still necessary. Indeed, it is still necessary, as demonstrated by the example in appendix B.

iPI (alg. 4) has the same best-case and worst-case runtimes as nPI (alg. 3), by similar arguments.

Algorithm 4: Improved Policy Iteration

```
1  Function iPI(Θ, s_I, action order 𝔸)
2      V(s) ← [[s ∈ S_F]] ∀s ∈ S
3      repeat
4          seen ← ∅
5          saw-unsafe ← false
6          rec-iPI(s_I)
7          if ¬saw-unsafe or V(s_I) = 1 then
8              return V(s_I)
9  Function rec-iPI(s)
10     // Rec-iPI has access to iPI's vars
11     if V(s) = 1 then
12         saw-unsafe ← true
13         return unsafe
14     if s ∈ seen then
15         return maybe-safe
16     seen ← seen ∪ {s}
17     // Look for safe action in 𝔸(s)
18     for a ∈ 𝔸(s) do
19         for s' ∈ supp(s, a) do
20             s'-status ← rec-iPI(s')
21         if s'-status ≠ unsafe ∀s' ∈ supp(s, a) then
22             return maybe-safe
23     // All actions unsafe means s unsafe
24     V(s) ← 1
25     saw-unsafe ← true
26     return unsafe
```

4 Experiments

We empirically compare iPI with JEA's TarjanSafe for deciding state safety: given a learned policy π that we wish to check for safety, we find a state s in π's graph, and interrogate iPI and TarjanSafe whether s is safe. Recall that finding such states is at the core of JEA's pipeline for testing the safety of the learned policy. We consider iPI with three different action orderings (1) *app*: the order of the model (same as TarjanSafe); (2) *learn*: using the learned action policy that we are evaluating; (3) *rand*: randomised. We do not consider Prop-$\mathcal{U}$ because it runs out of memory (see section 3.2 and appendix A). Our implementation extends JEA's C++ code and is available at (Schmalz and Jain 2026). All experiments were run on Intel Xeon E5-2660 CPUs with 12GB memory and a timeout of 15 minutes.

Benchmarks. We use JEA's benchmarks (scaled up), consisting of non-deterministic variants of *blocksworld* and *transport*; deterministic control benchmarks (*bouncing ball*, *follow car* and *inverted pendulum*); as well as different variants of a transport-like domain called *one/two-way line*, where a truck moves uni-(respectively bi-)directionally on a line, carrying packages to the right. We also introduce a new benchmark, *Flappy Bird*. In this problem, a bird must navigate a 2D environment from left to right, avoiding stationary obstacles. It can move up or down, and in each move may non-deterministically move to the right. Once the bird reaches the right side of the environment, it gets reset to the left side, i.e., it moves through the environment indefinitely unless it hits an obstacle. Thus, a safe policy moves the bird through the environment indefinitely, avoiding all obstacles. When safe policies exist, this showcases TarjanSafe's bad performance, taking inspiration from the example in fig. 1 in a more practical setting. Following JEA, all problems are encoded in JANI (Budde et al. 2017). For learned policies to test, we trained neural networks via Q-learning with two hidden layers of size 64.

Results. Our results are summarised in fig. 2. We note the benchmarks are solved in small timescales: the iPI variants solve almost all problems within 1 sec. This is not because the problems are small: we handle instances with up to 80 integer variables bounded by 70. To motivate this further, Prop-$\mathcal{U}$ runs out of memory as it enumerates all exponentially-many fail states (w.r.t. JANI description); similarly, Prop-$\mathcal{U}$ with regression over Binary Decision Diagram (BDDs), which can represent a set of states compactly, also runs out of memory as the BDDs become too large. Rather, the benchmarks are solved quickly by exploiting the features of state-avoiding problems. For deciding safe states, there are often cycles that TarjanSafe and iPI use to quickly construct safe policies that avoid fail states. For deciding unsafe states, TarjanSafe and iPI do not need to explore deeply before finding a way to propagate unsafety to s_I.

Comparing the variants of iPI: iPI(*app*) dominates the others. The orderings themselves do not appear to have a

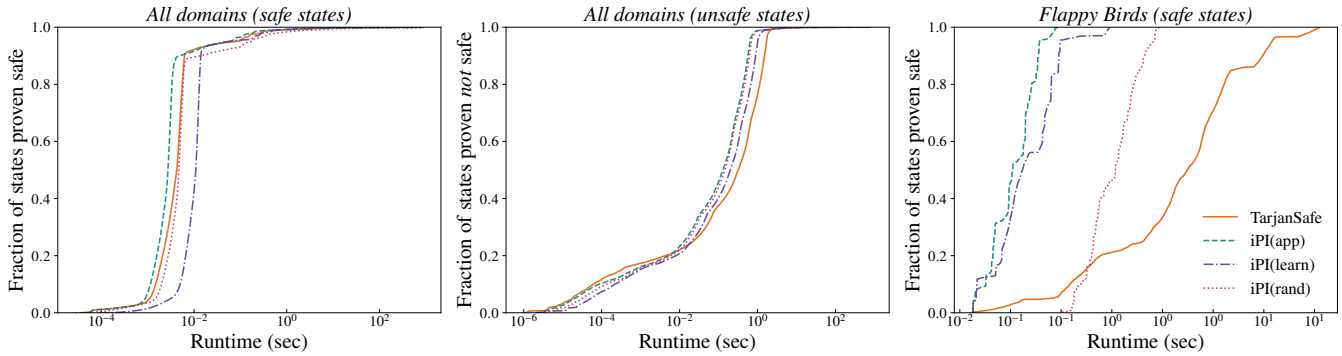

Figure 2: Cumulative plots of the proportion of states decided w.r.t. time. The plots separate safe and unsafe states: (left) is an aggregation over all domains' safe states, (middle) is over all unsafe states, and (right) is over safe states in Flappy Bird.

significant impact: all orderings tend to guess safe policies quickly (for safe states) and propagate the unsafe region to s_I efficiently (for unsafe states). Rather, the performance difference comes from implementation, namely the overhead of sorting with *learn* and randomising with *rand*.

Comparing iPI to TarjanSafe: For deciding states over all domains (left and middle), there is no significant difference between the approaches. JEA's benchmarks contribute most to these results, and they tend to be amenable to TarjanSafe, so this demonstrates that iPI indeed has similar best-case behaviour to TarjanSafe. In contrast, for deciding safe states in Flappy Birds, iPI scales exponentially better than TarjanSafe (note the runtime scale is logarithmic). This confirms our theory that iPI is exponentially faster than TarjanSafe in its worst case. So, iPI never performs significantly worse than TarjanSafe, and in some cases performs exponentially better.

Summary. These results confirm our theory that iPI(*app*) dominates TarjanSafe in the sense that it is similar for problems amenable to TarjanSafe, and exponentially better in problems that are problematic for TarjanSafe such as Flappy Bird. Furthermore, the results show that within iPI, the default action ordering performs best in our setting. We give a breakdown over domains and more discussion in appendix C, these are consistent with our findings here.

5 Conclusion

The state-safety decision problem is significant, being a necessary component of JEA's safety assurance pipeline for learned policies. We considered JEA's TarjanSafe algorithm for deciding state safety, and introduced Prop-$\mathcal{U}$ and iPI. We showed TarjanSafe has an exponential worst-case runtime but a good best case; in contrast, Prop-$\mathcal{U}$ has a linear worst case but impractical best case; iPI fills the gap, combining a polynomial worst case while maintaining the same best case as TarjanSafe. We confirmed this empirically: iPI is comparable to TarjanSafe when TarjanSafe does well, and otherwise iPI scales exponentially better. This places iPI as the current best algorithm for deciding state safety in our framework, and as the most promising starting point for future work.

Acknowledgements

Funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – GRK 2853/1 "Neuroexplicit Models of Language, Vision, and Action" - project number 471607914.

A Propagating Unsafety with Binary Decision Diagrams

Rather than regressing unsafety directly in the state space, it is natural to try to exploit problem structure and regress using Binary Decision Diagrams (BDDs). In JEA's setting, the set of fail states S_f is compactly represented as a symbolic formula ϕ_F called the **fail condition**. The symbolic representation of the unsafe region is equivalent to the weakest precondition a state has to satisfy to guarantee reaching the fail condition. This can be computed using regression (Rintanen 2008) and BDDs are state-of-the-art structures for such computations. A concrete implementation is provided in alg. 5.[4]

This algorithm uses ρ to represent the previously found unsafe region and τ for the newly found unsafe states. In each iteration, ρ is updated by adding the states in τ. τ is found by computing the preimage of each action with respect to ρ. Here, each action consists of a **guard** (g): to represent the weakest precondition a state must satisfy for the action to be applicable in it; and **updates** $(u_1, \dots u_n)$: to represent the nondeterministic effects of applying an action on a state. The following describes the preimage computation for a given set of successor states $S(v')$ and an action a. The set of predecessor states that can reach $S(v')$ on applying a is: $g(v) \wedge \exists v'(S(v') \wedge (u_1(v,v') \vee \dots u_n(v,v')))$. Intuitively, $\exists v'.S(v') \wedge (u_1(v,v') \vee \dots u_n(v,v'))$ is a relational product (Burch et al. 1994) to represent states where applying a leads to states in $S(v')$. A disjunctive partitioning ensures any nondeterministic effect leading to $S(v')$ is included. Conjuncting with $g(v)$ filters out all states where this action is not applicable.

[4]BDDs are restricted to boolean variables, but our transition system deals with linear constraints on integer variables, so we encode the constraints in *toBDD* as in Bartzis and Bultan (2006) .

Algorithm 5: Compute the unsafe region using Binary Decision Diagrams

```
1  Function computeUnsafe(φ) → BDD:
2      ρ ← toBDD(⊥); τ ← toBDD(φ);
3      while τ ⊈ ρ do
4          ρ ← ρ ∨ τ;
5          τ ← toBDD(⊤);
6          G ← toBDD(⊥);
7          foreach a ∈ A do
8              τ ← τ ∧ TR(a, ρ);
9      return ρ;
10 Function TR(a, ρ) → BDD:
11     if a = ⟨g, u_0|...|u_n⟩ then
12         return toBDD(g) ∧ (RelProd(toBDD(u_0), ρ) ∨
               ··· ∨ RelProd(toBDD(u_n), ρ))
```

Prop-$\mathcal{U}$ is not practical for our problems. The fail condition ϕ_F describes exponentially-many fail states S_f, so alg. 2 runs out of memory in its first step of enumerating fail states, even before attempting to propagate the unsafe region. With BDDs (alg. 5), it is possible to represent ϕ_F compactly, but after propagating it a few steps to describe the unsafe region the BDDs explode in size, again running out of memory. Thus, it seems that regressing the unsafe region is not a suitable approach to solve our problems.

B Is the Main Loop in iPI Necessary?

Is the main loop (lines 3–8) necessary, or is a single call to rec-iPI(s_I) enough? The loop is indeed necessary, because there are cases where rec-iPI(s_I) constructs an unsafe policy π and does not return unsafe. Let us apply iPI to the state-avoiding task shown in fig. 3, the key steps are:

- suppose rec-iPI first traverses $s_I, a_0, s_1, a_1, s_2, a_2$ and then encounters s_2 which is marked as `seen`, so the recursion unwinds to s_1
- at s_1, rec-iPI sees that the only action a_1 inevitably encounters a fail state, so it sets $V(s_1) = 1$ and backtracks to s_I,
- at s_I rec-iPI selects a'_0 as a potentially safe alternative to a_0 — this leads to s_2 which is in `seen`, now the recursion fully unwinds.

However, the policy π with $\pi(s_I) = a'_0$, $\pi(s_1) = a_1$, and $\pi(s_2) = a_2$ is unsafe. The key insight is that rec-iPI only propagates that states are unsafe with $V(s) = 1$ backwards, but in the presence of cycles, as we see in fig. 3, it may also be necessary to propagate this information forwards to detect that the policy is unsafe.

C Detailed Results

We break down the plots from fig. 2 by their domains in fig. 4 and fig. 5. Over domains, the results are consistent with the aggregated results in section 4, i.e., iPI(*app*) and TarjanSafe have similar performance on JEA's benchmarks and an exponential difference when deciding safe states on Flappy Birds; within iPI, the action ordering *app* generally outperforms *learn* and *learn*.

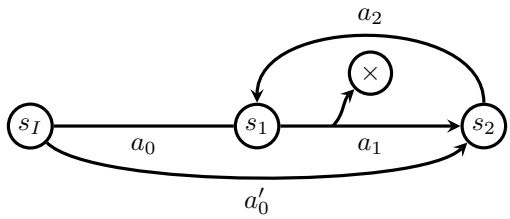

Figure 3: A state-avoiding task where a single call to rec-iPIfinds an unsafe policy but does not recognise it as unsafe.

We emphasise that TarjanSafe's worst case is only triggered by Flappy Bird when deciding *safe states*, and not when deciding *unsafe states*. This is because, for unsafe states, s_I can be proven unsafe before all paths need to be enumerated, bringing the task closer to TarjanSafe's best case. Consequently, there is no significant difference between iPI(*app*) and TarjanSafe when deciding unsafe states for Flappy Bird, and this does not contradict any of our claims.

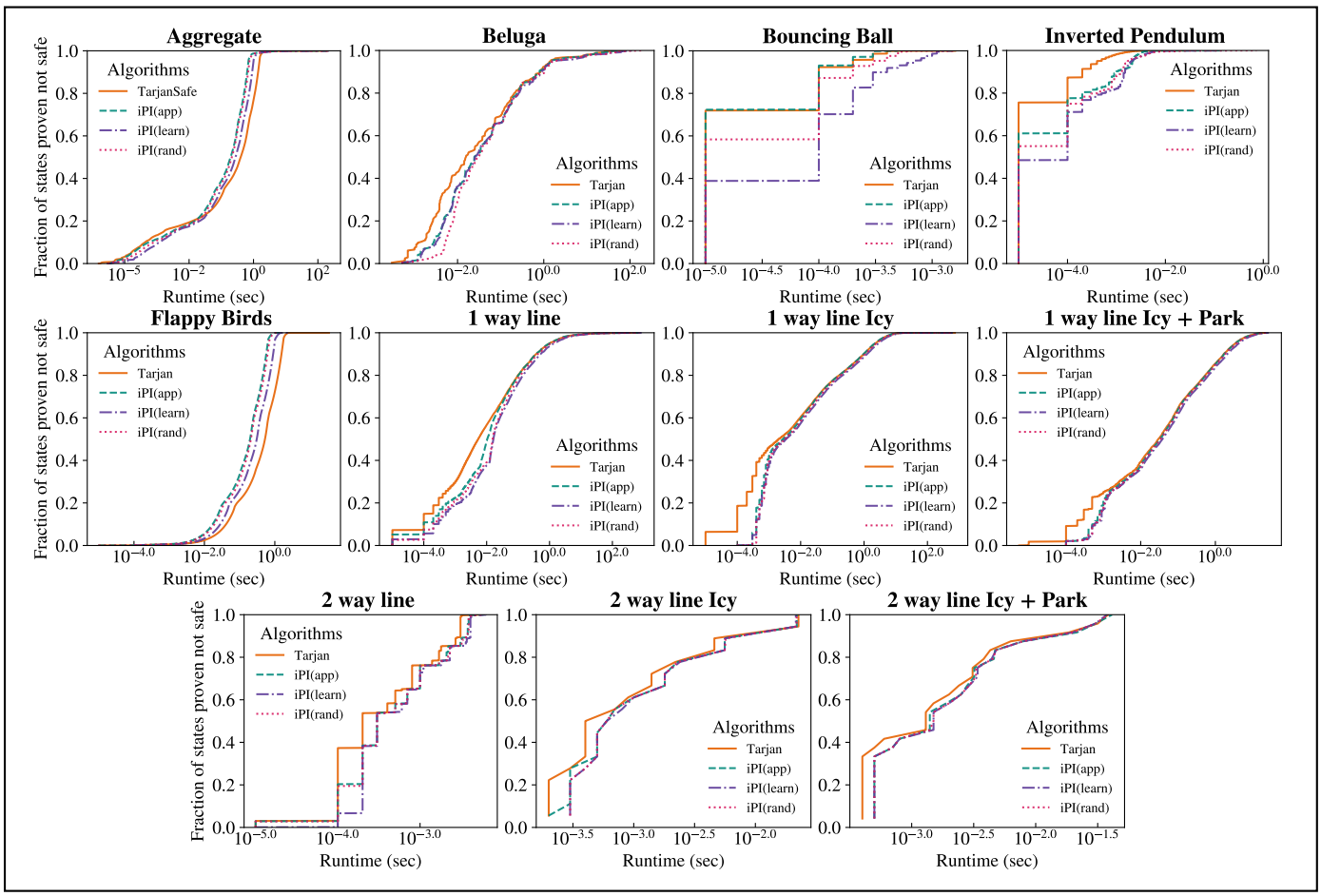

Figure 4: Cumulative plots of the proportion of states decided w.r.t. time for **unsafe states**.

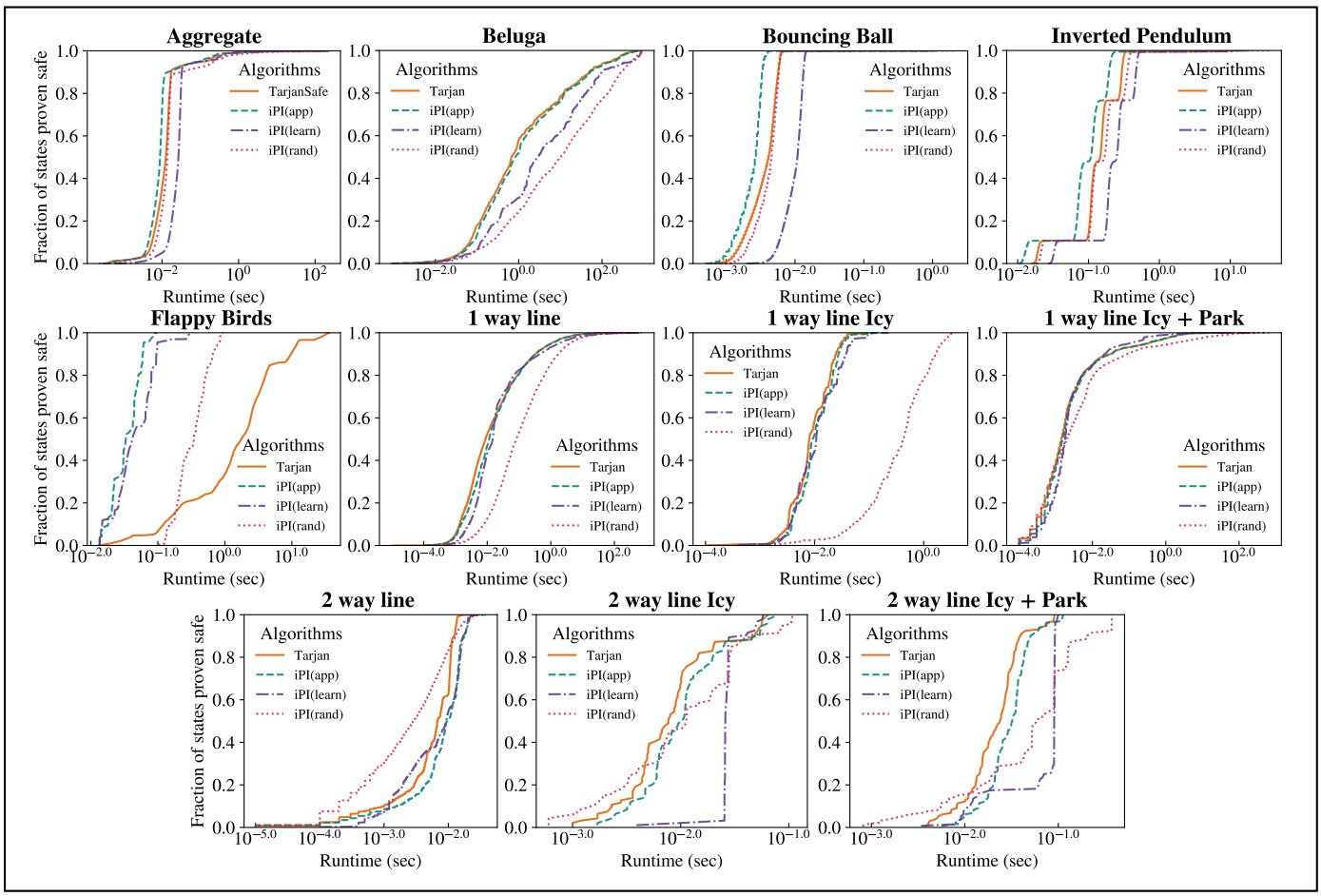

Figure 5: Cumulative plots of the proportion of states decided w.r.t. time for **safe states**.

References

Bartzis, C.; and Bultan, T. 2006. Efficient BDDs for Bounded Arithmetic Constraints. *Int. Journal on Software Tools for Technology Transfer*, 8(1): 26–36.

Bonet, B.; and Geffner, H. 2003. Labeled RTDP: Improving the Convergence of Real-Time Dynamic Programming. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 12–21.

Budde, C. E.; Dehnert, C.; Hahn, E. M.; Hartmanns, A.; Junges, S.; and Turrini, A. 2017. JANI: Quantitative Model and Tool Interaction. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 151–168.

Burch, J. R.; Clarke, E. M.; Long, D. E.; McMillan, K. L.; and Dill, D. L. 1994. Symbolic Model Checking for Sequential Circuit Verification. *IEEE Trans. on Computer-Aided Design of Integrated Circuits & Systems*, 13(4): 401–424.

Cimatti, A.; Pistore, M.; Roveri, M.; and Traverso, P. 2003. Weak, Strong, and Strong Cyclic Planning via Symbolic Model Checking. *Artificial Intelligence*, 147(1–2): 35–84.

Dai, P.; Mausam; Weld, D. S.; and Goldsmith, J. 2011. Topological Value Iteration Algorithms. *Journal of Artificial Intelligence Research*.

Daniele, M.; Traverso, P.; and Vardi, M. Y. 2000. Strong Cyclic Planning Revisited. In *Recent Advances in AI Planning*, 35–48.

Diekert, V.; and Kufleitner, M. 2022. Reachability Games and Parity Games. In *Int. Colloquium on Theoretical Aspects of Computing (ICTAC)*.

Hansen, E. A.; and Zilberstein, S. 2001. LAO*: A Heuristic Search Algorithm That Finds Solutions With Loops. *Artificial Intelligence*, 129(1): 35–62.

Howard, R. A. 1960. *Dynamic Programming and Markov Processes*. Technology Press of Massachusetts Institute of Technology.

Jain, C.; Sherbakov, D.; Vinzent, M.; Steinmetz, M.; Davis, J.; and Hoffmann, J. 2025. Policy Safety Testing in Non-Deterministic Planning: Fuzzing, Test Oracles, Fault Analysis. In *Proc. of European Conf. on Artificial Intelligence (ECAI)*.

Klauck, M.; Steinmetz, M.; Hoffmann, J.; and Hermanns, H. 2018. Compiling Probabilistic Model Checking into Probabilistic Planning. In *Proc. of the Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 150–154.

Mazala, R. 2002. *Infinite Games*, chapter 2, 23–38.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M. A.; Fidjeland, A.; Ostrovski, G.; Petersen, S.; Beattie, C.; Sadik, A.; Antonoglou, I.; King, H.; Kumaran, D.; Wierstra, D.; Legg, S.; and Hassabis, D. 2015. Human-Level Control Through Deep Reinforcement Learning. *Nature*, 518(7540): 529–533.

Muise, C.; McIlraith, S. A.; and Beck, J. C. 2024. PRP Rebooted: Advancing the State of the Art in FOND Planning. In *Proc. of AAAI Conf. on Artificial Intelligence*, 20212–20221.

Rintanen, J. 2008. Regression for Classical and Nondeterministic Planning. In *Proc. of European Conf. on Artificial Intelligence (ECAI)*, 568–572.

Schmalz, J.; and Jain, C. 2026. Code and Data for Algorithms for Deciding the Safety of States in Fully Observable Non-deterministic Problems. DOI: 10.5281/zenodo.18926835.

Silver, D.; Huang, A.; Maddison, C. J.; Guez, A.; Sifre, L.; van den Driessche, G.; Schrittwieser, J.; Antonoglou, I.; Panneershelvam, V.; Lanctot, M.; Dieleman, S.; Grewe, D.; Nham, J.; Kalchbrenner, N.; Sutskever, I.; Lillicrap, T.; Leach, M.; Kavukcuoglu, K.; Graepel, T.; and Hassabis, D. 2016. Mastering the Game of Go with Deep Neural Networks and Tree Search. *Nature*, 529: 484–503.

Silver, D.; Hubert, T.; Schrittwieser, J.; Antonoglou, I.; Lai, M.; Guez, A.; Lanctot, M.; Sifre, L.; Kumaran, D.; Graepel, T.; Lillicrap, T.; Simonyan, K.; and Hassabis, D. 2018. A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go Through Self-Play. *Science*, 362(6419): 1140–1144.

Tarjan, R. 1972. Depth-First Search and Linear Graph Algorithms. *SIAM Journal on Computing*, 1(2): 146–160.

Toyer, S.; Thiébaux, S.; Trevizan, F.; and Xie, L. 2020. ASNets: Deep Learning for Generalised Planning. *Journal of Artificial Intelligence Research*, 68: 1–68.